Aula 38: Deploy de Aplicações React

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

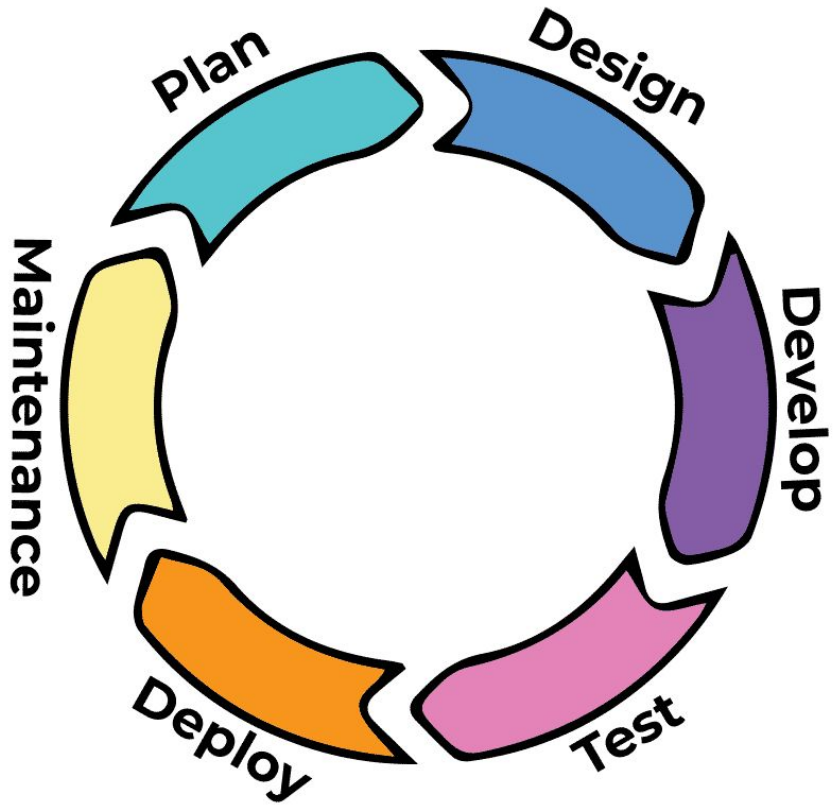
Índice

- INTRODUÇÃO
- BUILD DE PRODUÇÃO
- BUILD COM CRA
- BULD COM VITE
- CRA VS VITE
- HOSPEDAGEM ESTÁTICA
- DEPLOY COM NETLIFLY
- DEPLOY COM VERCEL
- NETLIFLY VS VERCEL
- BOAS PRÁTICAS
- EXERCÍCIOS

Introdução

DEPLOY

- Transição do ambiente de desenvolvimento (localhost) para o ambiente de produção (servidores acessíveis via URL)
- Processo: Código Fonte → Build (Empacotamento) → Hospedagem (Servidor)



Introdução

DEPLOY

Característica	Ambiente de Desenvolvimento	Ambiente de Produção
Otimização	Código legível, debug fácil	Código minificado, otimizado
Performance	Não prioritária	Alta performance, carregamento rápido
Variáveis de Ambiente	Locais (.env)	Gerenciadas pela plataforma

Build de produção

- Comando: npm run build
  - Transpilação: Converte JSX e ES6+ para JS compatível
  - Empacotamento: Agrupa módulos via Webpack
  - Minificação: Reduz o tamanho de arquivos JS e CSS
  - Cache Busting: Gera arquivos com hashes (ex: main.a1b2c3d4.js)
  - Saída: Cria a pasta build/ pronta para o servidor

Build de produção

- Ferramentas de Deploy Modernas
  - Hospedagem tradicional vs plataformas modernas
  - CI/CD (conceito introdutório)
- O que pode dar errado em produção?
  - Variáveis de ambiente
  - Rotas quebradas
  - Build incorreto

Build de produção

- Responsabilidade do Desenvolvedor
  - Testar antes de publicar
  - Garantir performance
  - Garantir segurança básica

Build com CRA

ESTRUTURA DO PROJETO

- src/
- public/
- package.json

SCRIPTS IMPORTANTES

- npm start
- npm test
- npm run build

Build com CRA

O QUE O BUILD FAZ

- Gera arquivos estáticos
- Otimiza o código
- Cria a pasta build/

PASTA BUILD

- index.html
- arquivos JS e CSS minificados
- hashes para cache

Build com CRA

LIMITAÇÕES DO CRA

- Build mais lento
- Menor flexibilidade
- Substituição gradual pelo Vite

Build com Vite

VITE

- Ferramenta moderna de build
- Foco em performance
- Dev server rápido
- Build otimizado
- Configuração simples

REALIZAR O BUILD

- npm run build
- Geração da pasta dist/

PASTA dist/

- index.html
- JS/CSS minificados
- assets com hash

Build com Vite

PREVIEW DE PRODUÇÃO

- Executa a versão de produção da aplicação
- Usa os arquivos gerados pelo build
- Não é o mesmo que o servidor de desenvolvimento
- Comando

```
npm run preview
```

Build com Vite

PREVIEW DE PRODUÇÃO

- O Preview simula:
  - Arquivos estáticos finais (dist/)
  - Comportamento real de rotas
  - Performance próxima da produção
  - Variáveis de ambiente de produção

Build com Vite

PREVIEW DE PRODUÇÃO

- Importância
  - Detecta erros antes do deploy
  - Reduz falhas em produção
  - Evita problemas de:
    - rotas quebradas
    - Caminhos relativos
    - variáveis de ambiente

Build com Vite

PREVIEW DE PRODUÇÃO

- Boas Práticas
  - Sempre rodar o preview antes do deploy
  - Conferir console do navegador
  - Testar navegação e recarregamento da página

CRA vs. Vite

Aspecto	create-react-app (CRA)	Vite
Status atual	Legado	Padrão moderno
Dev server	Webpack	ES Modules
Tempo de inicialização	Mais lento	Muito rápido
Build	Webpack	Rollup
Configuração	Abstraída	Simples e flexível
Pasta de saída	build/	dist/

CRA vs. Vite

QUANDO USAR CADA UM

- CRA
  - Projetos antigos
  - Manutenção de sistemas legados
  - Material didático mais antigo
- Vite
  - Projetos novos
  - Melhor experiência de desenvolvimento
  - Padrão do mercado atual

Exercício Prático

1. Crie um novo projeto React: npx create-react-app my-app
2. Explore a pasta build/ gerada e veja os arquivos estáticos.
3. Navegue até o diretório do projeto: cd my-app
4. Teste localmente com o pacote serve: npx serve -s build
5. Execute o comando de build: npm run build

Dica: Observe como os nomes dos arquivos JS e CSS mudaram após o build

Hospedagem Estática

- Por que não usar um servidor tradicional (VPS)?
  - Facilidade de Continuous Integration / Continuous Deployment (CI/CD)
- Plataformas consideradas
  - Netlifly
  - Vercel

Deploy com Netlifly

- Plataforma de hospedagem frontend para SPAs
- Foco em Edge Computing e facilidade de Drag & Drop
- Configuração de _redirects para Single Page Applications (SPA)

Deploy com Netlifly

VANTAGENS

- Integração Contínua: Conexão direta com GitHub, GitLab e Bitbucket
- CDN Global: Alta performance e baixa latência mundial
- HTTPS Automático: Certificados SSL gratuitos via Let's Encrypt
- Deploy Previews: Visualize mudanças em cada Pull Request

Deploy com Netlifly

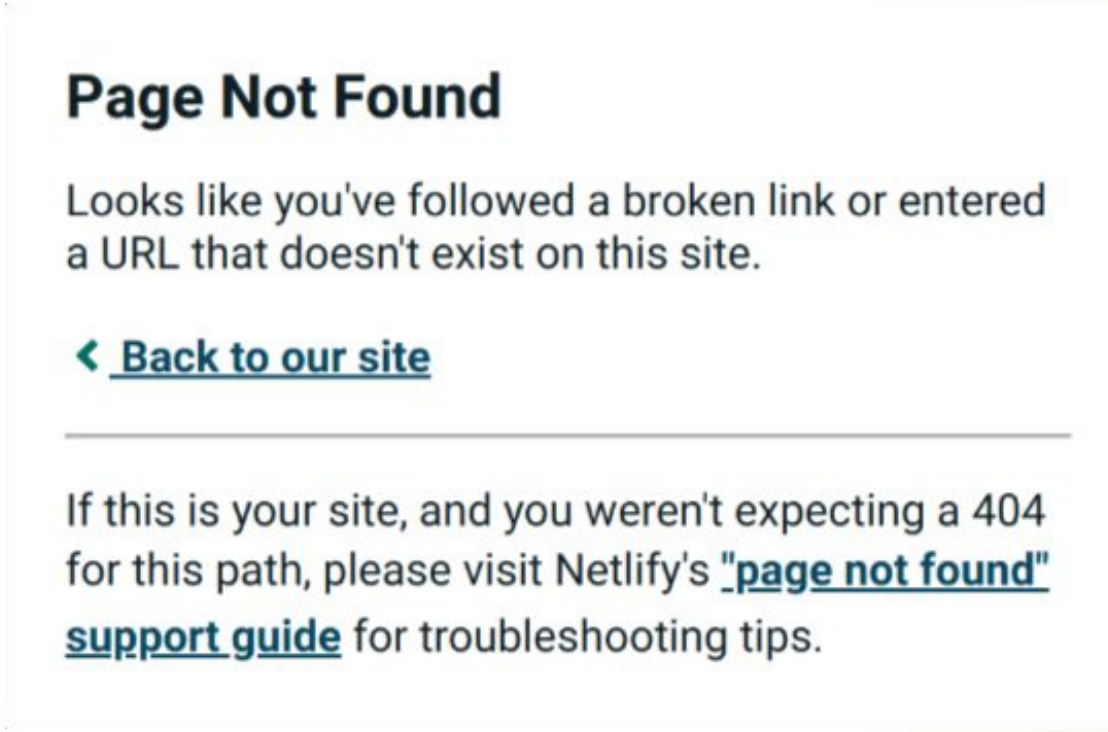
MÉTODOS DE DEPLOY

- Manual: Arraste e solte a pasta build/ no painel
- Via Git: Deploy automático a cada push na branch principal
- Netlify CLI: Comandos diretos do terminal para automação

Deploy com Netlifly

PROBLEMA DO 404

- Em SPAs, rotas como /sobre são virtuais
- Ao recarregar a página, o servidor busca um arquivo físico que não existe e retorna erro 404



Deploy com Netlifly

SOLUÇÃO

- Regra de Redirecionamento
  - Criar um arquivo chamado _redirects dentro da pasta public/
  - O arquivo deve conter:

    /* /index.html 200

  - Isso instrui o Netlify a servir o index.html para qualquer rota, permitindo que o React Router assuma o controle

Exercício

1. Crie um repositório Git para seu projeto React no GitHub/GitLab
2. Faça o push do código: git push origin main
3. No Netlify, selecione "Add new site" → "Import an existing project".
4. Conecte seu provedor Git e selecione o repositório criado.
5. Configure o build: npm run build e diretório de publicação: build.
6. Adicione o arquivo public/_redirects para garantir o funcionamento das rotas.
7. Clique em "Deploy site" e teste o link gerado!

Deploy com Vercel

- Plataforma focada em frontend
- Integração forte com React
- Conceito de "Preview Deployments" (um link para cada Pull Request)

Deploy com Vercel

VANTAGENS

- Foco em Performance: Criadores do Next.js, otimizada para front-end moderno
- Comparativo: Muito similar ao Netlify em recursos (Git, CDN, HTTPS)
- Zero Config: Detecção automática de frameworks como o CRA

Deploy com Netlifly

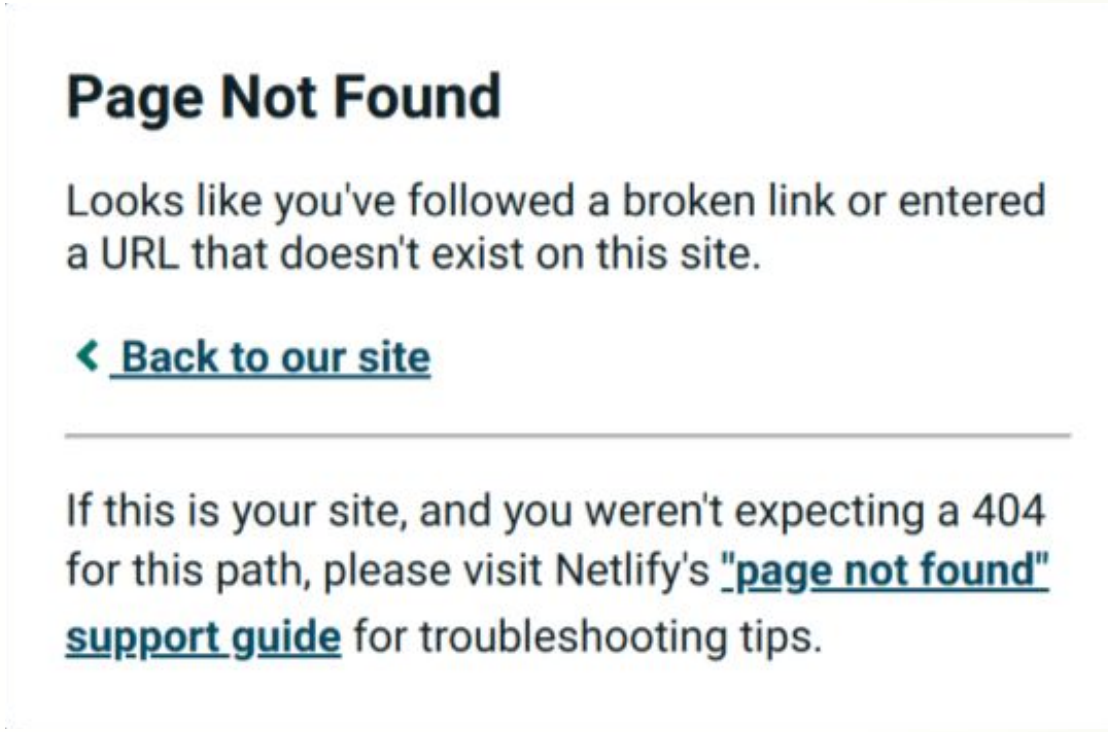
MÉTODOS DE DEPLOY

- Vercel CLI: Comando vercel para deploy instantâneo via terminal
- Via Git: Integração contínua com GitHub, GitLab e Bitbucket
- Instant Rollbacks: Volte para qualquer versão anterior com um clique

Deploy com Netlifly

PROBLEMA DO 404

- Assim como no Netlify, pode apresentar problemas de roteamento
- Rotas internas em SPAs precisam ser redirecionadas para o index.html para evitar erros ao recarregar a página



Deploy com Netlifly

SOLUÇÃO

- Crie um arquivo chamado vercel.json na raiz do seu projeto:
  - vercel.json
- Esta configuração instrui o Vercel a reescrever todas as requisições para o ponto de entrada da sua aplicação React

```
{
  "rewrites": [
    { "source": "/(.*)",
"destination": "/index.html" }
  ]
}
```

Exercício

1. Crie uma conta no Vercel e conecte seu repositório Git (GitHub/GitLab)
2. Selecione o projeto e verifique se o Vercel detectou o Create React App
3. Adicione o arquivo vercel.json na raiz do projeto com a regra de rewrites
4. Realize o deploy e teste as rotas da aplicação no domínio fornecido
5. Experimente o deploy via CLI: npm install -g vercel e depois vercel

Netlifly vs Vercel

FACILIDADE DE USO

- Netlify
  - Interface simples e didática
  - Ótimo para primeiros deploys
  - Configurações explícitas (build e publish)
- Vercel
  - Experiência “zero-config”
  - Integração muito forte com Git
  - Menos ajustes manuais

Netlifly vs Vercel

PERFORMANCE

- Netlify
  - CDN global
  - Excelente para sites estáticos e SPAs
  - Performance consistente
- Vercel
  - CDN otimizada para frontend moderno
  - Ótimo desempenho com React
  - Deploys e previews muito rápidos

Netlifly vs Vercel

PERFORMANCE

- Netlify
  - SPAs simples
  - Projetos educacionais
  - Sites estáticos e landing pages
- Vercel
  - Aplicações React modernas
  - Projetos com múltiplos deploys (preview)
  - Times que usam fluxo por branches

Boas Práticas

VERSIONAMENTO E HISTÓRICO

- Use Git como base do deploy
  - Commits pequenos e frequentes
  - Mensagens claras
  - Nunca fazer deploy sem versionamento
- Benefícios
  - Rastreabilidade
  - Facilidade de rollback
  - Trabalho em equipe

Boas Práticas

VARIÁVEIS DE AMBIENTE E SEGURANÇA

- Nunca expor dados sensíveis
  - Nunca salve chaves de API ou segredos no código
  - Tokens
  - URLs privadas
- Use arquivos .env com o prefixo REACT_APP_ e configure-os no painel do provedor (Netlify/Vercel)
- Nunca suba os .env para o GitHub
- Configurar variáveis no provedor de deploy

Boas Práticas

PERFORMANCE EM PRODUÇÃO

- Antes de publicar
  - Build otimizado (npm run build)
  - Remover logs desnecessários
  - Evitar dependências não utilizadas
  - Utilize Code Splitting para carregar apenas o necessário
- Objetivo
  - Carregamento rápido
  - Melhor experiência do usuário

Boas Práticas

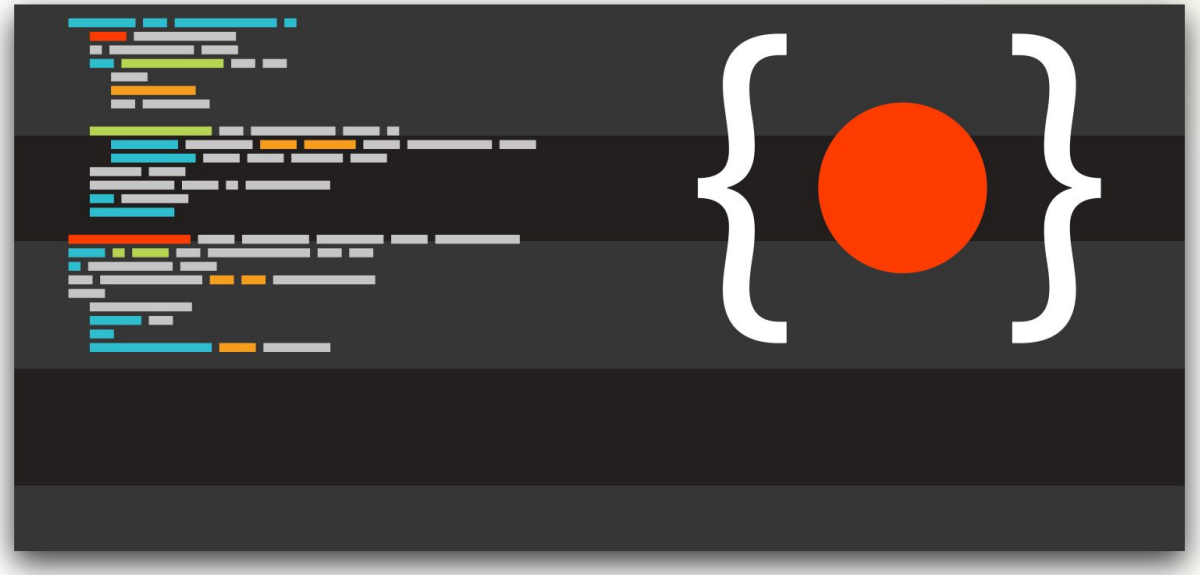
TESTES E VALIDAÇÃO

- Sempre validar antes do deploy
  - Testar localmente
  - Usar preview de produção
  - Verificar console do navegador
- O que observar
  - Erros
  - Rotas
  - Comportamento inesperado

Boas Práticas

MONITORAMENTO E LOGS

- Acompanhe a saúde da aplicação através dos Logs de Build e ferramentas de Analytics integradas



Exercício: Gerenciador de Tarefas

- Ao longo deste curso, vamos desenvolver progressivamente uma aplicação React completa, aplicando os conceitos estudados em cada aula
- O projeto será construído de forma incremental: A cada novo conteúdo, novas funcionalidades deverão ser adicionadas
- Ao final do curso, você teremos:
  - Uma aplicação React funcional
  - Um projeto organizado, testado e otimizado
  - Um deploy público em ambiente de produção

Exercício: Gerenciador de Tarefas

- O sistema consiste em uma aplicação web SPA (Single Page Application) para gerenciamento de projetos e tarefas, permitindo que o usuário:
  - Visualize projetos cadastrados
  - Acesse tarefas associadas a cada projeto
  - Crie, edite, conclua e remova tarefas
  - Navegue entre diferentes páginas da aplicação
  - Tenha seus dados persistidos localmente

Exercício: Gerenciador de Tarefas

- A aplicação deverá conter, no mínimo, os seguintes componentes:
  - App
  - Header
  - ProjectList
  - TaskList
  - TaskItem
  - TaskForm

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Deploy de Aplicações React**
  - Rodar o comando npm run build
  - Analisar a pasta dist (ou build) gerada
  - Subir para a Vercel ou Netlifly (via Git)
  - Configurar redirecionamento para o React Router não dar erro 404 ao atualizar a página

Exercício: Gerenciador de Tarefas

- **Entregas da aula de hoje: Deploy de Aplicações React**
- Conceitos implementados:
  - Build da aplicação para produção
  - Deploy no Netlify ou Vercel
  - Verificação do funcionamento em produção
- Entrega: O aluno acessa o site e envia o link pelo classroom